

AI Website Cloud Deployment with AWS

Development & Implementation Phase —DLBSEPCP01

By Fatima Ezzahrae Ezzouzi • 09-03-2026

Tutor: **Ahmet Nasri**

Bachelor: Applied AI

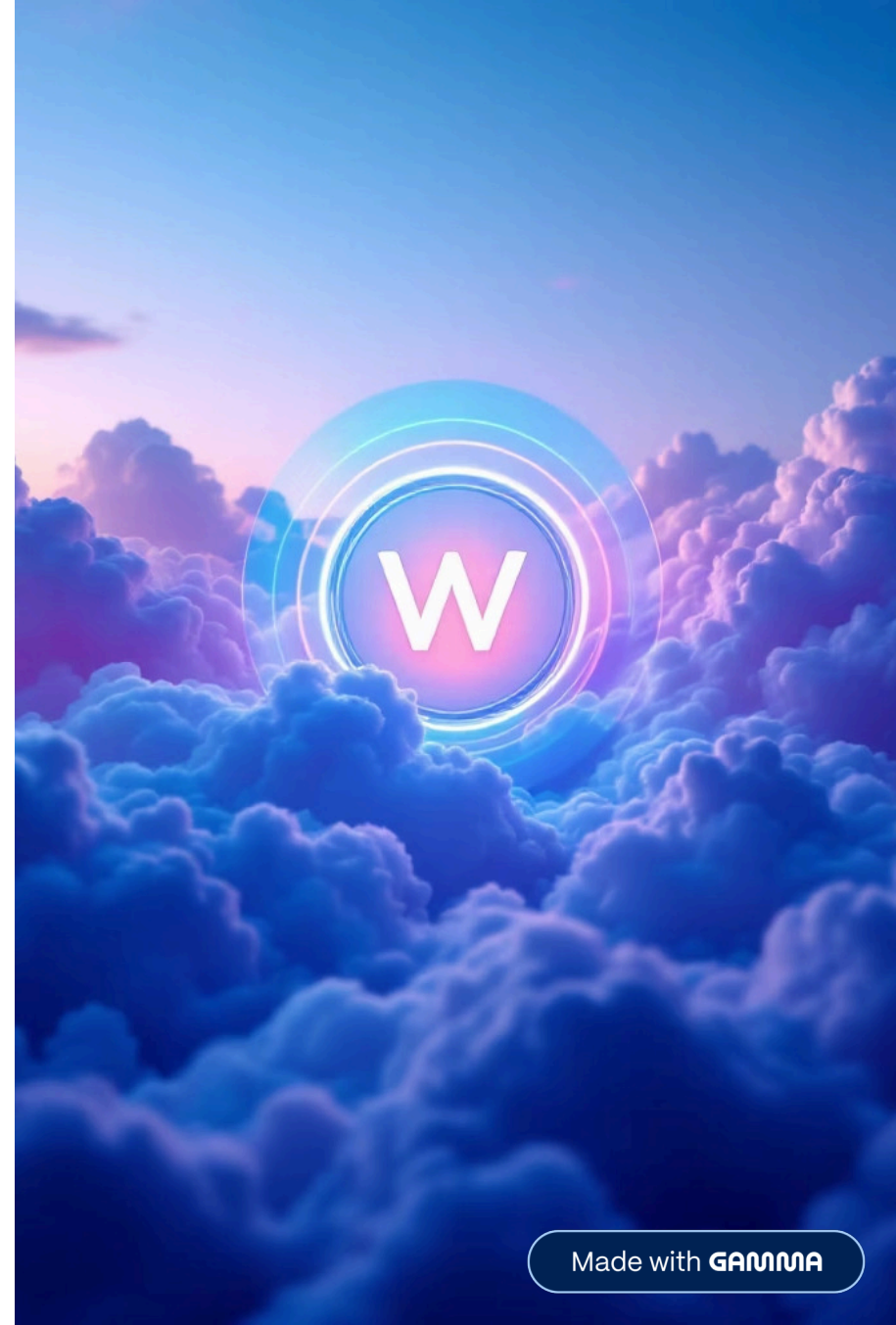


Table of content

- objective
- Tools and services
- Cloud architecture
- Terraform implementation
- Variables
- S3 bucket set up
- Cloudfront set up
- Outputs & testing
- Reflection
- References



	Objectives
	Architecture Diagrams
	Terraform Code
	CDN Configuration
	Testing Results



Objective

Host a static website on AWS via Amazon

Reliable object storage for site files and assets with durable hosting.

Use Terraform for automation

Infrastructure as Code to provision, version, and replicate the environment.

Global access & HTTPS via CloudFront

CDN-backed delivery with TLS termination and automatic HTTP→HTTPS redirects.

Tools & Services

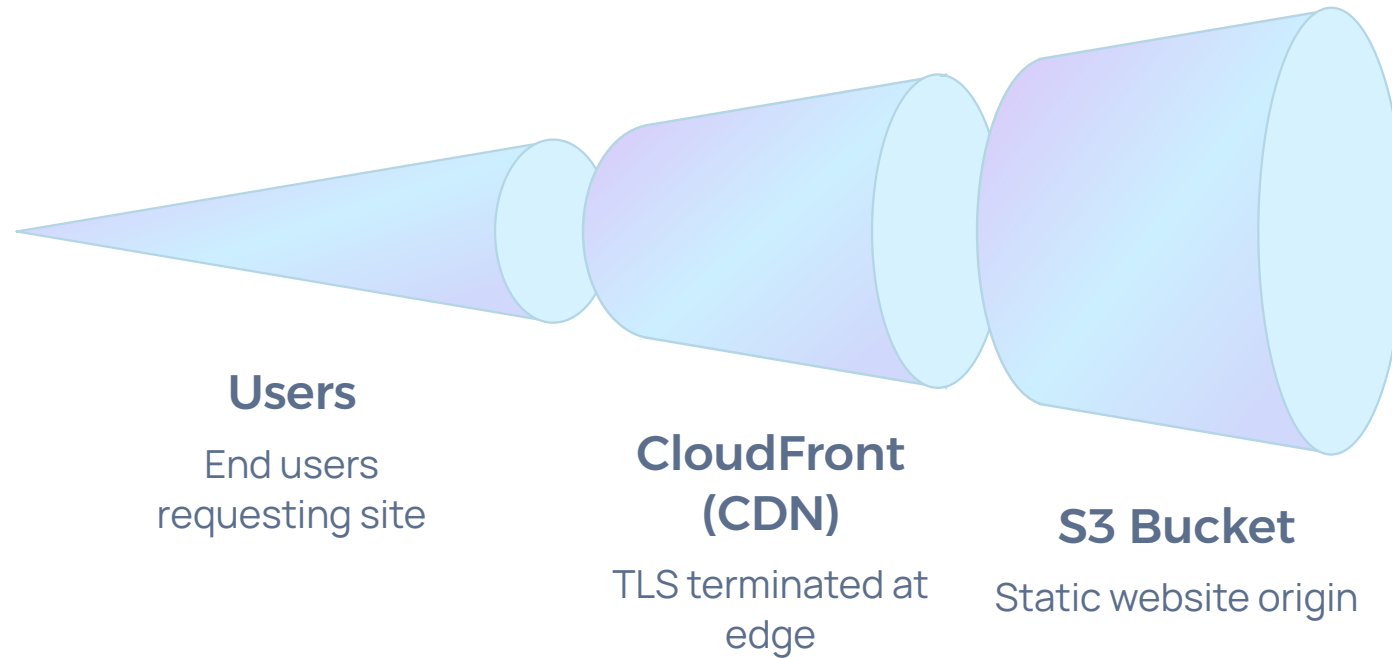


Key components used for the deployment:

Service / Tool	Purpose
AWS S3	Store website files & host static website – origin for CloudFront
AWS CloudFront	Global CDN, TLS termination, edge caching & redirects
Terraform	Infrastructure as Code: create S3, CloudFront, policies, and uploads

References: <https://aws.amazon.com/s3/> • <https://aws.amazon.com/cloudfront/> • <https://www.terraform.io/>

Cloud Architecture



User requests are routed to CloudFront edge locations (TLS terminated at edge). CloudFront fetches static content from the S3 website origin when not cached, serving the site globally with low latency.

Terraform Implementation

What terraform provisions

- S3 bucket for website files and hosting configuration
- CloudFront distribution with origin pointing to S3
- Bucket policy and OAI/Origin access configuration or public policy
- Optional automation to upload site assets

```
provider.tf
1
2  # 1 Set AWS Provider
3  provider "aws" {
4      region = var.aws_region
5  }
```

AWS PROVIDER

Set up AWS credentials :

- **AWS Access Key:** Identify an account
- **AWS Secret Key:** The corresponding secret key pairs with an access key

Set an AWS access key and secret key as an environmental variable

- **AWS_ACCESS_KEY_ID:** **YOUR_ACCESS_KEY**
- **AWS_SECRET_ACCESS_KEY:** **"YOUR_SECRET_KEY"**

variables

Defines all configurable values in one place:

```
variable "bucket_name" { default = "fatima-ai-website-2026" }  
variable "aws_region" { default = "us-east-1" }  
variable "project_name" { default = "AI-Website-CDN" }  
variable "index_document" { default = "index.html" }  
variable "origin_id" { default = "s3-website-fatima-ai" }
```

Benefits:

- Clean and reusable.
- Easy to override per environment (like dev, prod, or different projects).

```
variables.tf  
1  variable "aws_region" {  
2      description = "AWS region for deployment"  
3      type       = string  
4      default    = "us-east-1"  
5  }  
6  
7  variable "bucket_name" {  
8      description = "S3 bucket name"  
9      type       = string  
0      default    = "fatima-ai-website-2026"  
1  }  
2  
3  variable "project_name" {  
4      description = "Project tag name"  
5      type       = string  
6      default    = "AI-Website-CDN"  
7  }  
8  
9  variable "index_document" {  
0      description = "Main index file name"  
1      type       = string  
2      default    = "index.html"  
3  }  
4  
5  variable "origin_id" {  
6      description = "CloudFront origin ID"  
7      type       = string  
8      default    = "s3-website-fatima-ai"  
9  }
```

S3 Bucket Setup

```
resource "aws_s3_bucket" "website" {
  bucket        = var.bucket_name
  force_destroy = true
}

# Public Access Block
resource "aws_s3_bucket_public_access_block" "public_access" {
  bucket = aws_s3_bucket.website.id

  block_public_acls       = false
  block_public_policy     = false
  ignore_public_acls     = false
  restrict_public_buckets = false
}

# Bucket Policy for Public Read
resource "aws_s3_bucket_policy" "public_read" {
  bucket = aws_s3_bucket.website.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Principal = "*"
        Action = ["s3:GetObject"]
        Resource = "${aws_s3_bucket.website.arn}/*"
      }
    ]
  })
}

# Static Website Config
resource "aws_s3_bucket_website_configuration" "website_config" {
  bucket = aws_s3_bucket.website.id

  index_document {
    suffix = var.index_document
  }
}
```

- Created an S3 Bucket: fatima-ai-website-2026

- **Access**

Bucket policy set for public read or served via CloudFront origin access pattern.

- **Static website hosting**

Index document: index.html – configured in bucket properties.

```
# Upload index.html
resource "aws_s3_object" "index" {
  bucket        = aws_s3_bucket.website.id
  key           = var.index_document
  source        = var.index_document
  content_type  = "text/html"
}
```

- **Files uploaded**

index.html pushed via Terraform

CloudFront Setup

01

Create distribution

Origin: S3 website endpoint or origin access identity. Default root object: index.html.

Enable HTTPS

Use default CloudFront certificate (*.cloudfront.net) or ACM for custom domain.

```
resource "aws_cloudfront_distribution" "website_cdn" {  
  
  origin {  
    domain_name = aws_s3_bucket.website.website_endpoint  
    origin_id   = var.origin_id  
  
    custom_origin_config {  
      http_port      = 80  
      https_port     = 443  
      origin_protocol_policy = "http-only"  
      origin_ssl_protocols = ["TLSv1.2"]  
    }  
  }  
  
  enabled          = true  
  is_ipv6_enabled  = true  
  default_root_object = var.index_document  
}
```

```
viewer_certificate {  
  cloudfront_default_certificate = true  
}  
  
restrictions {  
  geo_restriction {  
    restriction_type = "none"  
  }  
}  
  
tags = {  
  Name = var.project_name  
}
```

01

Redirects & caching

Configure viewer protocol policy to redirect HTTP → HTTPS and **Cache behavior** (default_cache_behavior)

- `allowed_methods = ["GET", "HEAD"]` → Only allow read requests.
- `cached_methods = ["GET", "HEAD"]` → Cache these requests.
- `viewer_protocol_policy = "redirect-to-https"`
→ Forces users to use HTTPS (secure).

```
default_cache_behavior {  
  allowed_methods = ["GET", "HEAD"]  
  cached_methods = ["GET", "HEAD"]  
  target_origin_id = var.origin_id  
  
  viewer_protocol_policy = "redirect-to-https"  
  
  forwarded_values {  
    query_string = false  
    cookies {  
      forward = "none"  
    }  
  }  
}
```

•

SSL certificate

- `viewer_certificate { cloudfront_default_certificate = true }`
→ Uses CloudFront default HTTPS certificate.

Restrictions

- `geo_restriction { restriction_type = "none" }`
→ Website accessible from all countries.

Tags

→ Helps identify the resource.



Outputs & Testing

cloudfront url

CloudFront URL

- **What it is:** The **HTTPS domain name** of the CloudFront distribution
- **Purpose:** The public URL users use to access your content through CloudFront — primary global endpoint for testing

s3_url

S3 website endpoint

- **What it is:** The **S3 static website endpoint URL**
- **Purpose:** The direct URL where the S3 bucket content is hosted as a static website

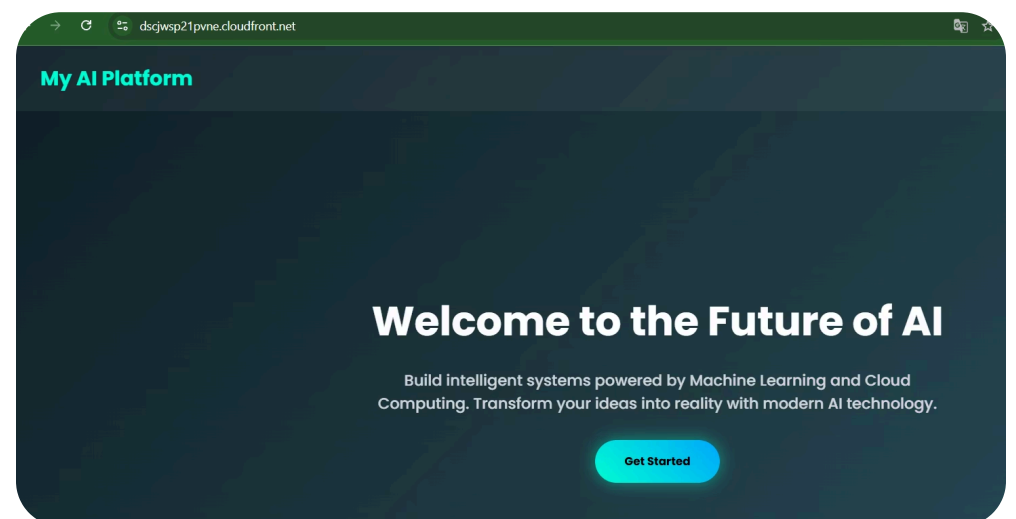
cloudfront id

- **What it is:** The unique **ID of the CloudFront distribution**
- **Purpose:** Used to reference or manage the CloudFront distribution in AWS

```
outputs.tf
1  output "cloudfront_url" {
2    description = "CloudFront URL"
3    value      = "https://${aws_cloudfront_distribution.website_cdn.domain_name}"
4  }
5
6  output "s3_url" {
7    description = "S3 Website URL"
8    value      = aws_s3_bucket_website_configuration.website_config.website_endpoint
9  }
10
11 output "cloudfront_id" {
12   description = "CloudFront Distribution ID"
13   value      = aws_cloudfront_distribution.website_cdn.id
14 }
```

— > Using the output links , we can have access to the webpage

the result (deployment of webpage)



Reflection

What we learned

Deploying a static site with S3 + CloudFront provides a cost-effective, globally-distributed delivery model. Terraform enables repeatable infrastructure.

Challenges

Configuring CloudFront TLS, managing origin access (OAI vs public), and Terraform resource dependency ordering were the main hurdles.

Solutions

Used CloudFront's default certificate for quick HTTPS, followed AWS & Terraform docs, and added explicit `depends_on` or data sources to clarify dependencies.



References

Primary references: <https://aws.amazon.com/s3/> • <https://aws.amazon.com/cloudfront/> • <https://www.terraform.io/>

